

Phase 2 — Multi-Provider Streaming Search Design

Date: 2026-05-07 **Status:** Approved (post-brainstorm) **Parent decomposition:** Phase 2 of 6 (see docs/CONTINUATION.md §4.1) **Constitutional bindings:** §6.J, §6.L, §6.R, §6.G, §6.E, §6.Q, §6.S, §6.T, §7

1. Problem Statement

Phase 1 shipped auth + security foundations. Five providers are registered in `lava-api-go` and six on the Android side, but only `rutracker` and `rutor` have working search end-to-end. The `α-hotfix` (commit `384ac02`) hid Internet Archive and other unsupported providers from the user-facing list via `apiSupported = false`. The operator's "alice-bug" report — "Search 'alice' on Internet Archive returns 'Something went wrong'" — is a symptom of this gap.

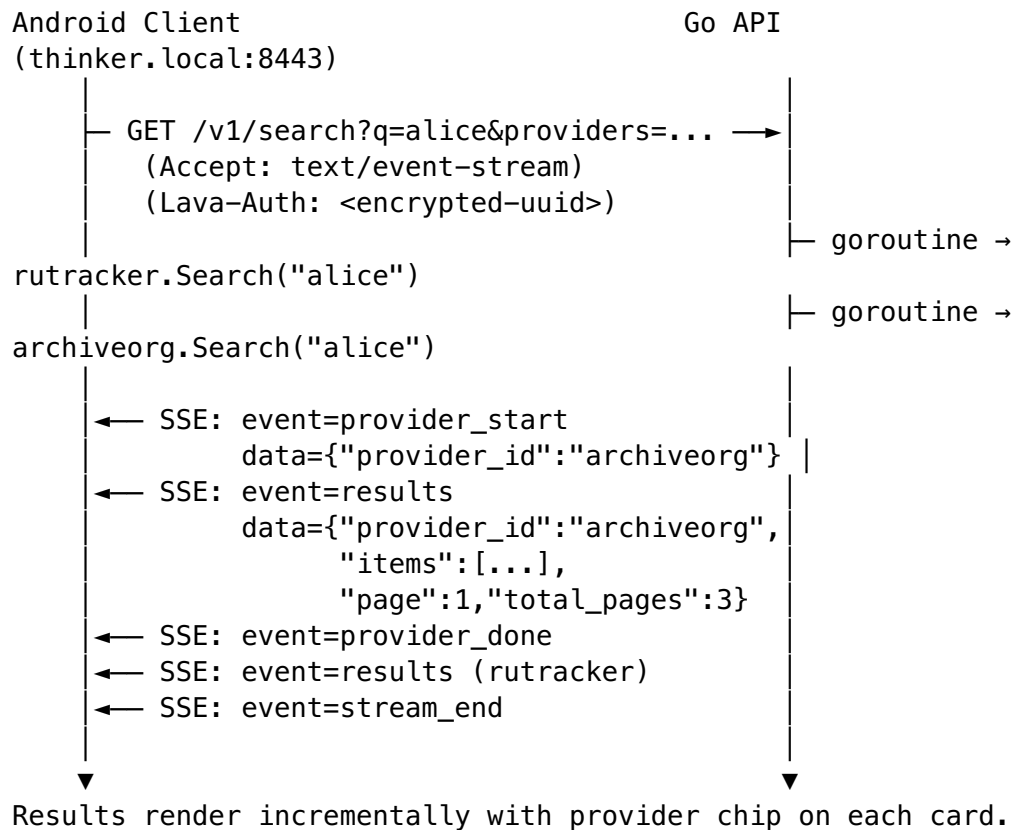
Phase 2 re-enables hidden providers by implementing their Go API scraper backends, wiring multi-provider search into the Android client with real-time streaming, and adding provider attribution to search results.

2. Design Decisions (from brainstorm)

Decision	Choice	Rationale
Streaming transport	SSE (Server-Sent Events)	Standard HTTP, unidirectional, no upgrade handshake, works over HTTP/3. Simpler than WebSocket or gRPC for one-way result streaming.
Search endpoint	Single aggregated / <code>v1/search</code>	Backend fans out to providers, streams results via single SSE connection. Cleaner client code, less connection overhead.
Provider label on results	Compact chip/badge on each card	Small colored chip in card corner. Subtle, scannable, uses existing design system.
Provider selector	Expandable chip bar below search input	Horizontal scrollable row of toggle chips. "All" chip + one per provider. Material 3 filter chip pattern.
Re-enablement order	archiveorg + gutenberg first , then <code>nnmclub</code> + <code>kinozal</code>	Anonymous providers (no auth) are simpler to implement. Fixes alice-bug fastest. Auth-requiring providers follow.

3. End-to-End Data Flow

User types "alice" → picks providers [rutracker, archiveorg] → taps Search



4. Go API Changes

4.1 New multi-search endpoint

GET /v1/search?
q=<query>&providers=<id1>,<id2>,...&page=1&sort=date&order=desc

Parameters:

- q (required): search query string
- providers (optional): comma-separated provider IDs. If empty or absent, defaults to ALL registered providers with SEARCH capability
- page, sort, order, categories, author, period: forwarded to each provider's Search method

4.2 New handler

File: lava-api-go/internal/handlers/v1/search.go — add or replace with MultiSearchHandler.

```

type MultiSearchHandler struct {
    Registry provider.Registry
}

func (h *MultiSearchHandler) GetMultiSearch(c *gin.Context) {
    query := c.Query("q")
    providerIDs := parseProviderList(c.Query("providers"))
    // defaults to all SEARCH-capable providers if empty

    c.Header("Content-Type", "text/event-stream")
    c.Header("Cache-Control", "no-cache")
    c.Header("Connection", "keep-alive")

    c.Stream(func(w io.Writer) bool {
        // Fan out to providers in goroutines
        // Stream SSE events as each completes
        // Return false when all done or client disconnects
    })
}

```

4.3 SSE event types

Event	Data shape
provider_start	{"provider_id":"archiveorg","display_name":"Internet Archive","total_providers":3}
results	{"provider_id":"archiveorg","items":[...TorrentItemDTO...],"page":1,"total_pages":3}
provider_error	{"provider_id":"kinozal","error":"upstream_timeout","message":"Ki did not respond within 30s"}
provider_done	{"provider_id":"archiveorg","total":67,"pages":3}
stream_end	{"providers_searched":3,"providers_failed":0,"total_results":301}

4.4 Provider scraper implementations

Phase 2a — anonymous providers:

- `internal/archiveorg/` — implement `Client` that:
 - Constructs search URL: `https://archive.org/advancedsearch.php?q=...&output=json`
 - Parses JSON response into `provider.SearchResult`
 - Maps archive.org metadata fields to `TorrentItemDTO` (identifier→id, title→title, etc.)
 - Handles pagination via archive.org's `rows/page` parameters
 - Returns `provider.ErrNotFound` on empty results
- `internal/gutenberg/` — implement `Client` that:
 - Constructs search URL: `https://www.gutenberg.org/ebooks/search/?query=...`
 - Scrapes HTML results into `provider.SearchResult`
 - Maps Gutenberg fields (title, author, id, formats) to `TorrentItemDTO`
 - Handles pagination via Gutenberg's `start_index` parameter

Both already have `ProviderAdapter` stubs that compile against `provider.Provider`. Work is limited to the internal `Client` implementations.

Phase 2b — auth-requiring providers:

- `internal/nmclub/` — implement `Client` with `FORM_LOGIN` auth flow
- `internal/kinozal/` — implement `Client` with `FORM_LOGIN` auth flow

4.5 Error handling

- Provider timeout (configurable, default 30s) → emit `provider_error`, continue with other providers
 - Provider returns empty results → emit `provider_done` with `total: 0`
 - All providers fail → emit `stream_end` with `total_results: 0`, client shows "No results" state
 - Client disconnects mid-stream → cancel all in-flight goroutines via context
-

5. Android Client Changes

5.1 New models and extensions

`core/models/src/main/kotlin/lava/models/search/Filter.kt` — add field:

```
data class Filter(  
    // ... existing fields ...  
    val providerIds: List<String>? = null, // null = all  
        available  
)
```

core/tracker/api/src/main/kotlin/lava/tracker/api/model/
TorrentItem.kt — ensure trackerId field is present (already exists in SDK
models).

5.2 SSE client

New file: core/network/impl/src/main/kotlin/lava/network/impl/
SseClient.kt

```
interface SseClient {  
    fun connect(url: String, headers: Map<String, String>):  
        Flow<SseEvent>  
}  
  
sealed interface SseEvent {  
    data class Event(val type: String, val data: String) :  
        SseEvent  
    data class Error(val throwable: Throwable) : SseEvent  
    data object StreamEnd : SseEvent  
}
```

Implementation uses OkHttp with a streaming response body reader. Parses SSE
wire format (event:, data:, empty line delimiters). Emits typed events via Kotlin
Flow. Handles:

- Connection timeout → SseEvent.Error
- Premature stream close → SseEvent.Error + optional reconnect
- Cancellation (search re-submit) → closes underlying HTTP connection

5.3 Provider chip bar

New composable: feature/search_input/src/main/kotlin/lava/search/
input/components/ProviderChipBar.kt

[All] [RuTracker] [RuTor] [Archive.org] [Gutenberg] ←
horizontally scrollable

Behavior:

- Reads available providers from
LavaTrackerSdk.listAvailableTrackers().filter
{ it.apiSupported }
- "All" chip: toggled on by default. Tap selects all providers, deselects individual
chips.
- Individual chips: tapping any deselects "All". Multiple can be selected.
- providerIds list passed to SearchInputAction.SubmitClick →
Filter.providerIds
- Selection remembered in SearchInputViewModel state for the session

Visual: Material 3 FilterChip with selected state. Each chip uses provider's
assigned color when selected.

5.4 Provider labels on result cards

Modify: `core/ui/src/main/kotlin/lava/ui/component/TopicListItem.kt`

Add optional parameter:

```
fun TopicListItem(
    topicModel: TopicModel<out Topic>,
    providerLabel: ProviderLabel? = null, // NEW
    // ... existing params ...
)
```

ProviderLabel data class:

```
data class ProviderLabel(
    val providerId: String,
    val displayName: String,
    val color: Color,
)
```

Rendered as a small SuggestionChip or AssistChip in the card's top-right corner, using the provider's assigned color (Section 7). When absent (legacy path), no chip is rendered — backward compatible.

5.5 SearchResultViewModel streaming integration

Extend `SearchResultViewModel` to support the new SSE path:

```
// NEW state variants added to SearchResultContent:
sealed interface SearchResultContent {
    // ... existing: Initial, Empty, Unauthorized, Content ...
    data class Streaming(
        val items: List<TopicModel<Torrent>>,
        val activeProviders: List<ProviderStreamStatus>,
    ) : SearchResultContent
}

data class ProviderStreamStatus(
    val providerId: String,
    val displayName: String,
    val status: StreamStatus, // SEARCHING, RECEIVING, DONE, ERROR
    val resultCount: Int,
)
```

Flow:

1. `onCreate` checks if `filter.providerIds` contains any `apiSupported` providers
2. If yes → calls `observeSseSearch()` instead of `observePagingData()`
3. If no (legacy, all non-`apiSupported`) → falls back to existing `observePagingData()`
4. `observeSseSearch()`:
 - Builds SSE URL from Endpoint config

- Connects via SseClient
- Maps each SSE event to state updates:
 - provider_start → adds ProviderStreamStatus(SEARCHING) to Streaming.activeProviders
 - results → appends items to Streaming.items, updates status to RECEIVING
 - provider_done → marks status as DONE, updates resultCount
 - provider_error → marks status as ERROR, shows inline error chip
 - stream_end → transitions to SearchResultContent.Content with final merged list
- On connection failure → SearchResultContent.Error with retry action

5.6 LavaTrackerSdk extension

Add method to LavaTrackerSdk:

```
fun streamSearch(
    filter: Filter,
    providerIds: List<String>,
): Flow<SseEvent>
```

This is the entry point that:

1. Resolves the Go API base URL from config
2. Constructs the SSE request URL with query parameters
3. Delegates to SseClient.connect()
4. Returns the event flow for the ViewModel to consume

6. Provider Color Scheme

Each provider gets a consistent Material 3 tonal palette color used for chips, labels, and status indicators:

Provider	trackerId	Color	Hex	Rationale
RuTracker	rutracker	Blue	#1976D2	Default/primary, existing theme
RuTor	rutor	Teal	#00897B	Distinct from blue, complementary
Internet Archive	archiveorg	Amber	#F9A825	"Archive/library" warmth
Project Gutenberg	gutenberg	Deep Purple	#7B1FA2	Literary, distinct
NNM-Club	nnmclub	Red	#D32F2F	Russian tracker, bold
Kinozal	kinozal	Orange	#E64A19	Film-related, warm

Colors are defined in a centralized ProviderColors object in core/designsystem/ and consumed by chips and labels. Chips use filled tonal variant (background tinted, text colored).

7. Provider Re-enablement Plan

Phase 2a — Anonymous providers (this plan)

Step	What	Deliverable
1	Implement <code>archiveorg.Client</code> (scraper)	<code>internal/archiveorg/client.go</code> + tests
2	Implement <code>gutenberg.Client</code> (scraper)	<code>internal/gutenberg/client.go</code> + tests
3	Implement <code>MultiSearchHandler</code> with SSE	<code>internal/handlers/v1/search.go</code> + tests
4	Wire in <code>main.go</code>	Route registration
5	Flip <code>apiSupported = true</code> on <code>ArchiveOrgDescriptor</code> + <code>GutenbergDescriptor</code>	Kotlin descriptor updates
6	Implement <code>SseClient</code> on Android	<code>core/network/impl/SseClient.kt</code> + tests
7	Implement <code>ProviderChipBar</code> composable	<code>feature/search_input/components/</code>
8	Implement provider labels on result cards	<code>core/ui/component/TopicListItem.kt</code>
9	Wire <code>SearchResultViewModel</code> streaming path	<code>feature/search_result/SearchResultViewModel.kt</code>
10	Challenge Test C17 (archiveorg search)	Composable test on real emulator
11	Challenge Test C18 (gutenberg search)	Composable test on real emulator
12	Challenge Test C19 (multi-provider SSE)	Composable test: select 2+ providers, verify both label chips appear

Phase 2b — Auth-requiring providers (future plan)

Step	What	Deliverable
1	Implement <code>nmclub.Client</code> with <code>FORM_LOGIN</code>	<code>internal/nmclub/client.go</code> + tests
2	Implement <code>kinozal.Client</code> with <code>FORM_LOGIN</code>	<code>internal/kinozal/client.go</code> + tests
3	Flip <code>apiSupported = true</code> on their descriptors	Kotlin descriptor updates
4	Challenge Tests C20 (nmclub) + C21 (kinozal)	Composable tests with credential pass-through

8. What Stays Legacy

For backward compatibility and gradual migration:

- **Legacy /search endpoint** (rutracker-only, root-mounted) remains active for existing clients
 - **SearchServiceImpl** → **NetworkApi** path stays for providers with `apiSupported = false`
 - **ObserveSearchPagingDataUseCase** remains the code path for single-tracker paginated search
 - **CrossTrackerFallbackModal** (Phase 4) continues to work independently — it fires on mirror unavailability, not on provider routing
 - **LavaTrackerSdk** is the routing layer: `apiSupported` providers → SSE path, others → legacy path
-

9. Testing Strategy

Go API

Layer	What	Location
Unit — scraper	archive.org JSON response → <code>SearchResult</code>	<code>internal/archiveorg/client_test.go</code>
Unit — scraper	Gutenberg HTML fixture → <code>SearchResult</code>	<code>internal/gutenberg/client_test.go</code>
Unit — SSE	Event struct → wire format marshalling	<code>internal/handlers/v1/search_test.go</code>
Integration	GET <code>/v1/search</code> → SSE stream, validate events	<code>tests/e2e/v1_search_test.go</code>
Contract	Content-Type, event format, error handling	<code>tests/contract/v1_search_contract_test.go</code>
Parity	Compare archiveorg/gutenberg results against baseline	<code>tests/parity/</code>

Android

Layer	What	Location
Unit — SSE client	Parse event stream, handle disconnect/reconnect	<code>core/network/impl/src/test/</code>
Unit — ViewModel	State transitions on each SSE event type	<code>feature/search_result/src/test/</code>
Unit — ProviderChipBar	Chip selection/deselection logic	<code>feature/search_input/src/test/</code>

Layer	What	Location
Structure	ProviderChipBar layout regression (no nested scroll violation per §6.Q)	feature/ search_input/src/ test/
Challenge C17	archiveorg search: enter "alice" → select Archive.org → results appear with amber chip	app/src/ androidTest/
Challenge C18	gutenberg search: enter "sherlock" → select Gutenberg → results appear with purple chip	app/src/ androidTest/
Challenge C19	multi-provider: select RuTracker + Archive.org → search → both chips visible on respective cards	app/src/ androidTest/

All tests carry Bluff-Audit stamps per Seventh Law clause 1. All Challenge Tests exercise real Compose UI on the §6.I emulator matrix.

10. Files Affected (summary)

Go API

File	Change
lava-api-go/internal/handlers/v1/search.go	Add MultiSearchHandler with SSE streaming
lava-api-go/internal/archiveorg/client.go	Implement real scraper
lava-api-go/internal/gutenberg/client.go	Implement real scraper
lava-api-go/cmd/lava-api-go/main.go	Wire multi-search route (or extend existing register)
lava-api-go/api/openapi.yaml	Document new /v1/search endpoint + SSE event types

Android

File	Change
core/models/.../search/Filter.kt	Add providerIds field
core/network/impl/.../SseClient.kt	New — SSE client
core/designsystem/.../ProviderColors.kt	New — per-provider color definitions
core/ui/.../TopicListItem.kt	Add providerLabel parameter, render chip
core/tracker/client/.../LavaTrackerSdk.kt	Add streamSearch() method

File	Change
feature/search_input/.../ components/ProviderChipBar.kt	New — multi-select chip bar
feature/search_input/.../ SearchInputViewModel.kt	Wire provider selection into Filter
feature/search_result/.../ SearchResultScreen.kt	Wire streaming path
feature/search_result/.../ SearchResultViewModel.kt	Add SSE observation, new state variants
feature/search_result/.../ SearchPageState.kt	Add Streaming and ProviderStreamStatus types

11. Risks and Mitigations

Risk	Mitigation
archive.org JSON API changes format	Fixture-based tests catch parse regressions. Contract test against live API.
Gutenberg HTML structure changes	Fixture freshness check (30d warn, 60d block per scripts).
SSE connection unreliable on mobile networks	Reconnect with exponential backoff. Show inline error chip, don't fail whole search.
Streaming state complicates ViewModel	Separate observeSseSearch() path from observePagingData(). Clear branching, not interleaved.
Provider colors clash with dynamic themes (Phase 5)	Provider colors are assigned hex values, not theme aliases. Theme changes in Phase 5 pick them up via color reference.

12. Constitutional Compliance

- **§6.J / §6.L** (Anti-Bluff): every SSE event → UI state transition has a Challenge Test asserting user-visible outcome
- **§6.E** (Capability Honesty): providers only flip `apiSupported = true` after real scraper implementation exists + passes parity test
- **§6.G** (Provider Operational Verification): descriptor `verified` remains false until Challenge Test passes on emulator matrix
- **§6.R** (No-Hardcoding): all provider base URLs, timeouts, and colors come from config/generated code, never literals
- **§6.Q** (Compose Layout Antipattern Guard): `ProviderChipBar` uses `LazyRow`, never nested in `verticalScroll` parent
- **§6.S** (Continuation): this design doc referenced from `CONTINUATION.md`, updated per commit
- **§7** (Bluff-Audit): every test commit carries `Bluff-Audit:` stamp demonstrating deliberate break → test failure → revert